

Tarea Examen 2

Karla Romina Juárez Torres

27-marzo-2025

Problemas

0.1.

Diseñe, en pseudocódigo, un método que reciba una lista ligada L y una pila P , el cual deberá modificar la lista original eliminando las posiciones indicadas por cada nodo de la pila. Use las operaciones propias de cada estructura. Calcule la O .

Ejemplo: Si $L = \{2, 4, 6, 8, 9, 3\}$ y $P = \{2, 5\}$ el método regresa $L = \{2, 6, 8, 3\}$

```
Method eliminarPosiciones(L, P) {
    posLista = [];
    while (!P.isEmpty()) {
        pos = P.pop();
        posLista.add(pos);
    }

    // Ordena posLista en orden descendente

    posLista.orden_descendente();

    for (pos in posLista) {
        if (pos < 1 || pos > L.length()) {
            continue;
        }
        if (pos == 1) {
            L.head = L.head.siguiente;
        } else {
            actual = L.head;
            contar = 1;
            while (contador < pos - 1 & actual != null) {
                actual = actual.siguiente;
                contador += 1;
            }
            if (actual != null && actual.siguiente != null) {
```

```

        actual.siguiente = acutal.siguiente.siguiente;
    }
}
}
return L;
}

```

La extracción de las posiciones tiene una complejidad $O(m)$ donde m es el número de elementos en P , y el ordenamiento posList $O(m \log(m))$ y la eliminación de los elementos $O(m * n)$

Así la complejidad $O(m * n)$ domina $O(m \log(m))$

0.2.

2.1 ¿Cuánto tarda la ejecución del método que inserta un elemento en un hash?

$O(1)$ suponiendo que está bien diseñada y distribuye uniformemente las claves y en el peor de los casos tendría una inserción de $O(n)$

2.2 Describa la estrategia lineal para resolver una colisión. Si se utiliza esta técnica ¿cómo se garantiza el mismo tiempo de ejecución mencionado en el inciso anterior? El concepto recae en usar el siguiente espacio disponible en la table secuencialmente

$$\text{Nueva posición} = (h(k) + i) \text{ mód } m$$

tal que $h(k)$ es el valor hash original de la clave, i el numero de intentos y m el tamaño de la tabla

Ahora para garantizar el tiempo constante la se debe tener un factor de carga controlado, la tabla debe redimensionarse a un tamaño mayor, y con la busqueda de uan buena función hash que se distribuya uniformemente.

0.3.

Redefina el TDA de ListaLigada, de tal manera que garantice que el tiempo de ejecución de la operación size es $O(1)$.(Basta con reescribir sólo aquellas partes del TDA que se ven afectadas). Mencione las ventajas y desventajas.

Primeramente agregamos un atributo relacionado al tamaño que almacene el número de elementos actuales

```

class listaligada {
    private Node head;
    private int size;

    public Lista_ligada() {
        this.head = null;
        this.size = 0;
    }
}

```

Esto con el fin de obtener el tamaño directamente en $O(1)$ También tenemos podemos cambiar la inserción y la eliminación de incrementar en uno o viceversa

```
public void Insertar_final(int valor) {
    Node nuevo_nodo = new Node(valor);
    if (head == null) {
        head = nuevo_nodo;
    } else {
        Node actual = head;
        while (actual.siguiente != null) {
            actual = actual.siguiente;
        }
        actual.siguiente = nuevo_nodo;
    }
    size++;
}

public boolean eliminar(int valor) {
    Node actual = head;
    Node anterior = null;
    while (actual != null) {
        if (actual.data == valor) {
            if (anterior != null) {
                anterior.siguiente = actual.siguiente;
            } else {
                head = actual.siguiente;
            }
            size--;
            return true;
        }
        anterior = actual;
        actual = actual.siguiente;
    }
    return false;
}
```

Tenemos varias ventajas, claramente el tener un tiempo constante aumenta la eficiencia y su simpleza pero por otro lado el uso de memoria adicional puede ser problemático, y además se debe asegurar que size se actualiza para cada operación ya que si no sucede puede terminar con una inconsistencia.

0.4.

Redefina el TDA de Celda (que represente a la celda para una lista doblemente ligada), de manera tal que cada una tenga nombre y posición. Defina sus operaciones e incluya los axiomas necesarios para garantizar el correcto funcionamiento de la estructura.

Primeramente necesitamos una celda tal que contenga un nombre, posición, su anterior y siguiente.

```

    public Celda creador_celda(String nombre, int posicion) {
        Celda celda = new Celda();
        celda.nombre = nombre;
        celda.posicion = posicion;
        celda.anterior = null;
        celda.siguiente = null;
        return celda;
    }

```

de aqui solo necesitamos obtener cada caracteristica o establecerla tal que:

```

public int getCaracteristica(Celda celda) {
    if (celda == null) {
        error;
    } else {
        return celda.Caracteristica;
    }
}

```

```

public void setCaracteristica(Celda , Caracteristica) {
    if (celda == null) {
        error;
    } else {
        celda.Caracteristica = caracteristica;
    }
}

```

Ahora solo necesitamos verificar la consistencia con la posición, la integridad de los enlaces y que no se repitan los nombres.

```

public boolean Verificador_Consistencia(Celda celda) {
    if (celda.siguiente != null && celda.siguiente.posicion != celda.posicion)
        return false;
    }
    if (celda.anterior != null && celda.anterior.posicion != celda.posicion - 1)
        return false;
    }
    return true;
}

public boolean Validar_enlace(Celda celda) {
    if (celda.siguiente != null && celda.siguiente.anterior != celda) {
        return false;
    }
}

```

```

        if (celda.anterior != null && celda.anterior.siguiente != celda) {
            return false;
        }
        return true;
    }

    public boolean Verificar_Unico(Lista_doble_ligada , indicador) {
        Celda actual = list.head;
        while (actual != null) {
            if (actual.indicador.equals(indicador)) {
                return false;
            }
            actual = actual.siguiente;
        }
        return true;
    }
}

```